

How *models learn.*

Follow the learning loop. Watch a model improve.
Understand why it can still make mistakes.

🕒 20-25 min core reading · Beginner friendly · 12 short sections

A model learns by changing its numerical settings in response to a training signal. The useful question is whether those changes help on cases it has not learned from.

WHAT YOU WILL BE ABLE TO EXPLAIN

Trace a prediction through loss and a parameter update. Separate training, validation, and testing.

Recognize underfitting, overfitting, leakage, and changing data. Explain why a well-trained model can still give a wrong answer.

Reading path: Follow one small runtime example, try the training experiment, and complete three core exercises. The optional mathematics can wait. You need Chapters 1-2 and basic arithmetic.

1. Learning starts with a task and examples

Suppose an operations assistant must estimate a job's runtime **before the job starts**. The input is job size; the output is a number of seconds. Predicting a numerical value is a **regression** task. Predicting a category, as in Chapter 2, is **classification**.

A **dataset** is a collection of examples. Each row below describes one fictional completed job. The **feature** is information supplied to the model; the **label** or **target** is the answer it should learn to predict.

Training example	Feature x: size, in thousands of records	Target y: actual runtime, in seconds
A	1	3
B	2	5
C	3	7

These three rows are deliberately clean **synthetic data**, created to make the arithmetic visible. They are not enough to establish a useful production predictor. Real runtime also depends on hardware, competing work, and the kind of processing.

The prediction moment matters. Job size may be available before execution; the eventual finish time is not. A model trained with information unavailable at that moment cannot be evaluated fairly for the intended use.

2. A prediction becomes a measurable error

Choose a simple model:

$$\hat{y} = wx + 1$$

Read this as: **predicted runtime = weight × job size + 1 second**. The hat on y means “predicted.” The **weight w** is a parameter we will learn. Here the constant +1 is fixed; in a more general straight-line model, the intercept can also be learned.

Start at $w = 1$. The three predictions are 2, 3, and 4 seconds. Compare each with its actual value:

Example	Predicted	Actual	Error: predicted – actual	Squared error
A	2	3	–1	1
B	3	5	–2	4
C	4	7	–3	9

A **loss function** gives a numerical penalty for predictions. We will use **mean squared error (MSE)**: square each error, add the results, and divide by the number of examples.

$$\text{MSE} = \frac{1 + 4 + 9}{3} = \frac{14}{3} \approx 4.67 \text{ seconds}^2$$

Squaring prevents positive and negative errors from cancelling and gives larger errors more influence. The units are squared seconds, not seconds. At $w = 2$, the predictions become 3, 5, and 7, so this training MSE is zero.

That is a perfect fit to **these three rows**. It does not establish perfect performance on future jobs. Google's loss lesson explains common regression losses.

▼ Go deeper: loss, objective, and evaluation metric

A training **objective** is what the learning procedure tries to optimize. It can be the average loss alone or include other terms, such as a penalty discouraging excessive model complexity.

An **evaluation metric** is a measure used to judge the result. It can match the loss, but need not. Here, mean absolute error (MAE) at $w = 1$ is $(1 + 2 + 3) / 3 = 2$ seconds. MAE is easier to interpret in the original units; MSE gives large errors more weight.

The chosen metric must also reflect the task. A small average error might hide a few severely underestimated runtimes. Examine errors on large jobs and any cases that matter operationally. Classification and language modeling often use different losses; not every model is trained with MSE.

3. The training loop changes parameters

The model does not need a person to manually try every possible weight. An **optimizer** is a procedure for adjusting parameters to improve the objective.

➤ Learning changes the weight; prediction uses the weight

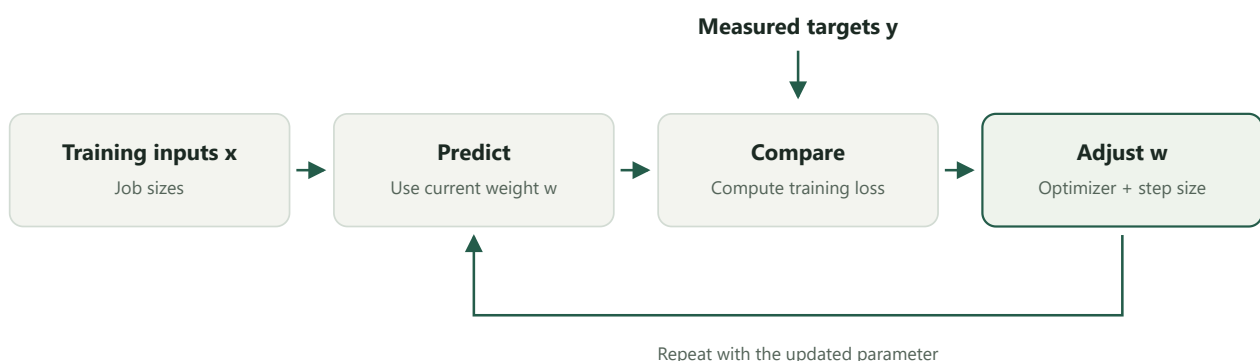


Figure 1. One training loop. The runtime target is used to measure loss, not supplied as an input feature for a new prediction.

One common optimizer is **gradient descent**. The **gradient** describes how the objective changes as parameters change. Gradient descent moves parameters in the opposite direction, with the step size controlled by a **learning rate**.

For our example, increasing w from 1 toward 2 reduces the training loss. An appropriate update moves in that direction. A step that is too large can cross the low-loss region and make the result worse.

The distinction is useful: **w is a learned parameter; the learning rate is a hyperparameter**, a setting of the learning procedure. [Google's gradient descent lesson](#) develops this update mechanism.

A **batch** is the set of examples used for one update. An **epoch** is one pass through the training set. Our tiny experiment uses all three examples per update, so one step is also one epoch here. With many batches, one epoch usually contains many steps.

Training repeatedly predicts, compares, and adjusts. **Inference** uses the fitted parameter to predict a new runtime. Reading that output, or supplying a new input, does not by itself update the weight.

▼ **Optional mathematics: calculate the first update**

For this particular dataset, the loss as a function of w is:

$$L(w) = \frac{14}{3}(w - 2)^2 \quad \frac{dL}{dw} = \frac{28}{3}(w - 2)$$

The derivative is the slope of the loss with respect to w . At $w = 1$ it is $-28/3$, approximately -9.3333 . A gradient descent update with learning rate 0.10 gives:

$$w_{\text{new}} = w - 0.10 \frac{dL}{dw} = 1 - 0.10(-9.3333) \approx 1.9333$$

The new training MSE is approximately 0.0207 squared seconds. A fresh start with rate 0.25 instead gives $w \approx 3.3333$ and MSE ≈ 8.2963 : worse than the initial 4.6667.

The interface displays rounded values but calculates with full numerical precision. These learning rates make sense for this tiny, scaled dataset; they are not universal recommendations.

Neural networks extend the idea to many parameters. **Backpropagation** efficiently computes the gradients through the network; the optimizer uses those gradients to update parameters. We will derive this after the mathematical foundations. Some ML methods, such as ordinary decision-tree training, use other procedures.

4. Try one real training step

Press **Train one step** and watch the weight, line, and loss change. This is an actual numerical update of a tiny linear model in your browser.

Predict. Compare. Adjust.

The model predicts $\text{seconds} = w \times \text{job size} + 1$. Only w changes; the $+1$ stays fixed.

Figure 2. Training experiment, printable results. Each step uses the three training examples. The intercept stays at 1.

State	Weight w	Training MSE (s^2)
Start	1.0000	4.6667
One step at rate 0.10	1.9333	0.0207
Fit at $w = 2$	2.0000	0.0000
Fresh start, one step at 0.25	3.3333	8.2963

The larger rate overshoots and increases loss. At $w = 2$, the separate validation examples (1.5, 4.2) and (2.5, 5.8) have MSE 0.04 s^2 . They do not affect training updates.

Try three steps at 0.10, then switch to 0.25 for a fresh run. With the larger rate, the line alternates across the training points and the errors grow. The experiment stops if loss exceeds 100 squared seconds or after 20 steps; these are demonstration limits.

The separate validation examples deliberately contain small deviations from the straight line. At $w = 2$, they have MSE 0.04 even though training MSE is zero. A training score and a new-example score answer different questions.

Observe before continuing: Which value changed during training? Did opening the validation check change it? The validation check only measures the current model. Selecting a learning rate based on that check would be a development decision.

5. Give training, validation, and testing different jobs

Generalization means useful performance on relevant cases beyond those used to fit the model. To estimate it, keep some examples separate.

➤ Three datasets, three responsibilities

SPLIT BEFORE FITTING DATA-DEPENDENT PREPARATION

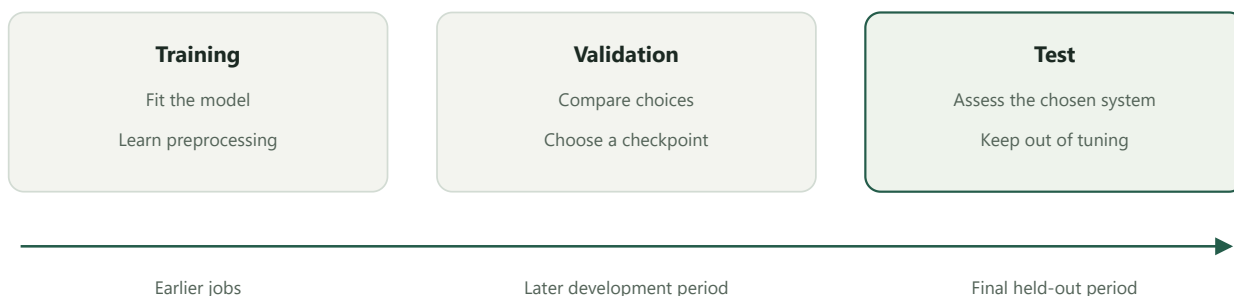


Figure 3. The roles matter more than a fixed percentage. Validation guides choices; the test set stays separate until the chosen system is assessed.

Training set: fit parameters. Any data-dependent preparation, such as learning feature averages for normalization, must also be fitted on the training portion.

Validation set: compare choices such as model size, learning rate, and when to stop. It influences development choices, even if it is not used in gradient updates.

Test set: assess the selected system after those choices are fixed. Repeatedly tuning to its results turns it into development data.

There is no universally correct split percentage. The split should imitate the use case. For future job prediction, train on earlier jobs and evaluate on later periods. If near-duplicate jobs from one run appear in both sets, apparent performance may be misleading.

Data leakage happens when information that should be unavailable enters training or model selection. Using a job's actual finish time as an input to a pre-start prediction is leakage. Fitting preprocessing on all examples before splitting can also leak information. [scikit-learn's common pitfalls guide](#) explains prevention.

▼ Go deeper: validation is useful, but not unlimited evidence

Trying many alternatives on one validation set can gradually overfit your choices to that set. A separate final test helps reveal this. Report the size and scope of the evaluation; two validation rows, as in our experiment, are only an illustration.

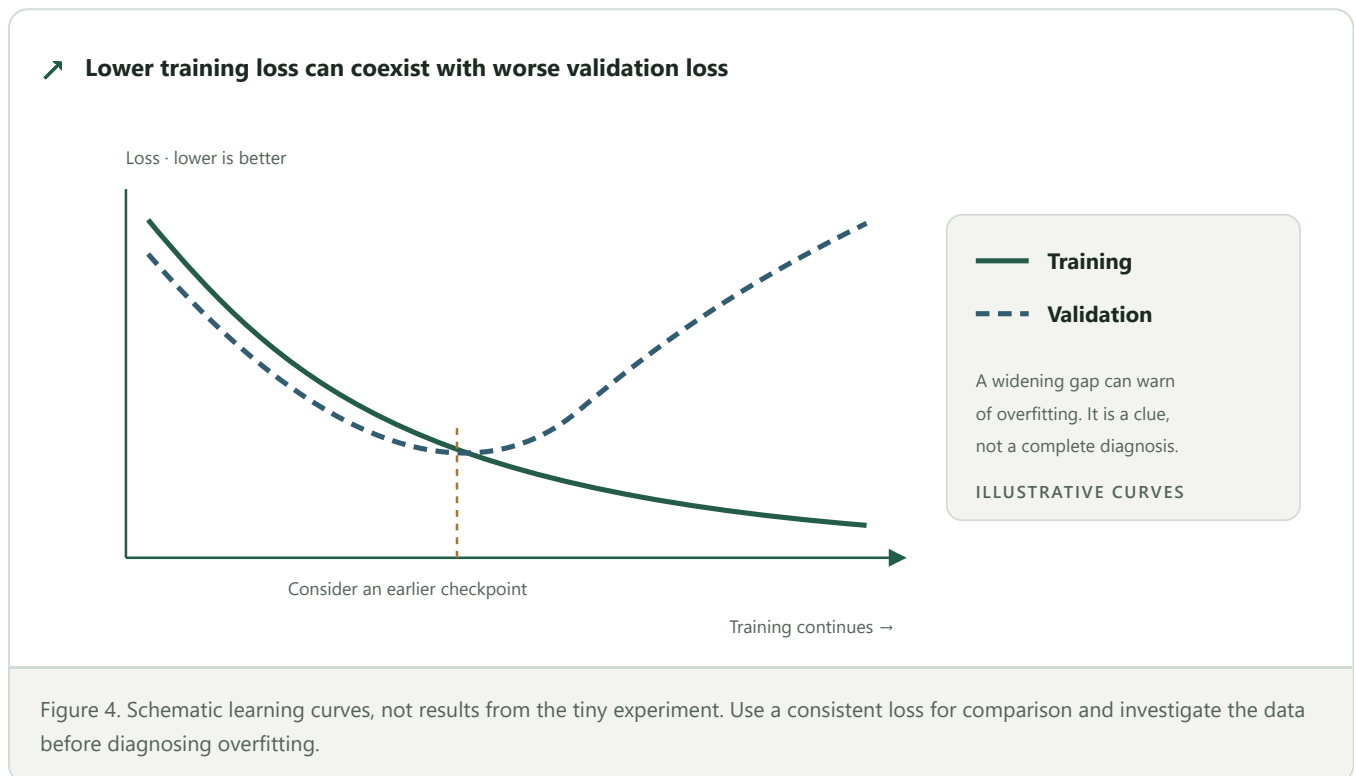
Cross-validation rotates which development examples are used for fitting and validation across several splits. It can reduce dependence on one split, but it does not excuse leakage or remove the need for an appropriate final evaluation.

Respect time and grouping during splitting. Several records from the same viewer, movie, or processing batch may be dependent. The split must match whether deployment will encounter new groups, future records from existing groups, or both.

6. Underfitting and overfitting

Underfitting means the fitted model misses useful structure. A constant runtime estimate may miss the relationship between job size and runtime. Both training and validation errors can remain high. Causes can include an unsuitable model, missing features, or insufficient optimization.

Overfitting means fitting peculiarities of the training examples that do not help on relevant new cases. A flexible model might memorize individual job IDs or noisy timing fluctuations. Its training error can be excellent while validation error is worse.



The graph shows one possible warning pattern: training loss keeps falling while validation loss starts rising. It is an illustration, not a law that every real learning curve must follow. A validation gap can also reflect different data distributions or a flawed split.

Possible responses include a simpler model, better or more representative data, **regularization** that discourages some kinds of complexity, and **early stopping** at a checkpoint selected using validation performance. More training is not automatically the cure. [Google's overfitting lesson](#) explains the distinction.

Do not diagnose from one score alone. Compare training and validation results, inspect individual mistakes, and check that the evaluation setup represents the intended task.

7. Why a model can still be wrong

Poor performance is not one problem with one fix. Use the error to decide what to investigate.

What you observe	Possible explanation	A useful check
Similar jobs have different runtimes	Measurement noise or missing information, such as server load	Check measurement definitions and available features
Good average results, poor results on large jobs	Those jobs may be rare in training	Compare error and sample count by job-size group
Excellent offline results, poor live results	Leakage or mismatched preparation	Reproduce exactly what was available at prediction time
Results worsen after a hardware change	Distribution shift: deployment data or relationships differ	Compare recent inputs and errors with the development period
The model uses a job name as a shortcut	A correlation may not remain useful outside the sample	Test across job families and relevant future periods

A relationship in data is not automatically causal. Larger jobs may often run overnight on faster machines. Changing the job size alone may not reproduce the association learned from that mixture.

Also distinguish estimation from certainty. Our line gives a point estimate, not a guarantee or an uncertainty interval. A plausible-looking number can still be wrong.

▼ Go deeper: bias and uncertainty are not single ideas

In model fitting, **bias** can mean systematic error from restrictive assumptions; it can also mean the intercept parameter in a linear model. In discussion of people and decisions, bias often concerns unfair or unrepresentative behavior. State which meaning you intend.

Some uncertainty comes from limited knowledge or insufficient data. Some comes from variability not captured by the inputs. Better data or features may reduce error, but repeated observations need not produce identical outcomes.

For classification, a model's numeric score is not automatically a calibrated probability. For regression, an uncertainty interval needs its own method and coverage checks. Later evaluation chapters develop these ideas.

8. The learning signal can take different forms

The runtime example uses **supervised learning**: inputs are paired with target answers. The broader learning landscape includes other signals:

Approach	Where the signal comes from	Example
Supervised learning	Target values or categories supplied with examples	Job size paired with measured runtime
Self-supervised learning	Targets constructed from the data itself	Use earlier text to predict a later token
Unsupervised learning	Structure in data without supplied task labels	Group movie reviews by similar patterns
Reinforcement learning	Rewards from actions and outcomes during interaction	Learn a scheduling policy in a simulator

These categories are useful descriptions, not perfectly exclusive boxes. Self-supervised learning is often discussed within unsupervised learning, and a system can combine several approaches.

Reward also differs from a correct answer for every step. In reinforcement learning, a useful outcome can depend on a sequence of actions. The training setup must connect outcomes to those actions.

The common thread is an objective or learning signal shaping behavior. Success against that signal still needs to be checked against the actual task.

9. How this connects to LLMs and wrong answers

In a common language-model training setup, the target is the next token already present in a text example. The model estimates token probabilities; a loss penalizes assigning too little probability to the target. This is a self-supervised training signal. [Hugging Face's training overview](#) describes this approach.

Later training can use instruction examples or feedback about preferred responses. Those stages change the training data or objectives. They do not make every generated claim true.

A language model's learned patterns may not contain a current fact, may reflect an unreliable source, or may not be enough to resolve an ambiguous request. During generation it can produce a fluent continuation that is incorrect or unsupported: a **hallucination**.

For our movie assistant, missing streaming availability should remain unknown. A plausible service name is not established by fluent wording. Supplying a catalog entry through RAG changes the model's context; fine-tuning changes its parameters. Both still need evaluation.

The practical habit: name the symptom before changing the system. Missing documents suggest investigating retrieval. Incorrect arithmetic suggests using exact code. A misleading offline score suggests checking evaluation and leakage. Do not assume every failure requires a bigger model.

10. Practice: calculate, separate, diagnose

Exercise 1: check another weight

For the three training rows, use $w = 1.5$ and the same fixed $+1$. Calculate the predictions and MSE. Is the loss better than the initial $w = 1$? Does that establish good future performance?

▼ Reveal the calculation

Predictions are 2.5, 4, and 5.5 seconds. Errors are -0.5 , -1 , and -1.5 ; squared errors are 0.25, 1, and 2.25. $MSE = 3.5 / 3 \approx 1.1667$ squared seconds, lower than 4.6667. This establishes a better fit to these rows under MSE, not good performance on future jobs.

Exercise 2: find the leakage

A team predicts runtime before a job starts. Its dataset includes size, machine type, and the actual finish timestamp. It fits feature-normalization values on the whole dataset before creating a test split. Identify two problems and correct them.

▼ Reveal the explanation

The actual finish timestamp is future information, so remove it from pre-start inputs. Fitting normalization on the whole dataset lets test information affect preparation. Split appropriately first, fit preparation on the training portion, then apply that fitted transformation to validation and test data. Keep the final test separate from development choices.

Exercise 3: investigate the gap

After more training, training MSE drops from 2 to 0.2, while validation MSE rises from 3 to 6. What might be happening? Give two reasonable next checks or changes without using the test set to tune them.

▼ Reveal the explanation

This is an overfitting warning. First check the split and inspect validation errors, because the numbers alone do not establish the cause. Compare an earlier checkpoint or a simpler/regularized model using validation data. Use the final test after selecting the approach. More training without investigating the gap may make it worse.

▼ Extra practice: connect training with inference

Exercise 4. At $w = 2$, predict the runtime for 2.5 thousand records. If the observed runtime is 5.8 seconds, what is the squared error? Did making the prediction update w ?

▼ Reveal the calculation

The prediction is $2 \times 2.5 + 1 = 6$ seconds. Squared error is $(6 - 5.8)^2 = 0.04$ squared seconds. Neither making this prediction nor measuring its error changes w ; a training update would be a separate operation.

Exercise 5. A model performs well on old hardware and poorly on a new platform. Must it be overfitting?

▼ Reveal the explanation

No. The input-to-runtime relationship may have changed: distribution shift. Check hardware and workload differences, preprocessing consistency, and recent error cases before choosing a remedy. Several causes can coexist.

11. Check your understanding

Try all five questions, then use the explanations to revisit any uncertain distinctions.

01 For runtime prediction, which value is the label?

- A. The job size known before execution.
- B. The actual runtime measured after the job finishes.
- C. The learning rate chosen by the developer.

Answer: B. The label is the training target. Job size is an input feature; learning rate is a setting of the training procedure.

02 A model has zero loss on its training examples. What follows?

- A. It will make no mistakes on future jobs.
- B. Its training examples must cover every possible case.
- C. It fits those examples under that loss; new-case performance still needs evaluation.

Answer: C. Training loss measures fit to the training data. A flexible model can even memorize training examples without learning a useful general pattern.

03 Which is data leakage?

- A. Using a viewer's review written after watching to predict their enjoyment before watching.
- B. Keeping later jobs in a separate test period.
- C. Fitting a normalization rule on training data alone.

Answer: A. The later review contains future information unavailable when the recommendation must be made. An offline score using it would overstate what the deployed system could do.

04 Training error falls while validation error rises. What should you investigate?

- A. Whether validation examples were used for training.
- B. Overfitting; compare a simpler or earlier model on validation data.
- C. Whether the test set can be used to tune every parameter.

Answer: B. The widening gap is a useful overfitting warning. Examine split quality and error cases too; a curve alone is not a complete diagnosis. Keep final testing separate from model selection.

05 An LLM is given a newly retrieved movie entry. What normally changes?

- A. Its learned parameters are automatically updated.
- B. Its training objective is replaced.
- C. Its input context changes for that call.

Answer: C. Using new context is inference. Fine-tuning is a separate training process that updates parameters. A generated answer still needs evidence checks.

12. Keep the learning loop in mind

Explain the chapter in one connected sequence:

Choose a task and examples. Predict with a model. Measure a loss. Update parameters. Use validation to choose the approach. Test on separate relevant cases. Inspect mistakes after deployment.

Before moving on, explain why zero training loss is not a guarantee, why future information can invalidate an evaluation, and why a new prompt is different from a training update.

▼ Quick-reference glossary

Term	Recall cue
Feature / target	Input information / answer used as the learning target
Regression / classification	Predict a number / predict a category
Parameter / hyperparameter	Fitted model value / setting chosen for learning or model design
Loss / objective	Prediction penalty / quantity the procedure optimizes
Gradient / optimizer	Local change signal / procedure that uses it or another rule to update parameters
Learning rate	Controls the size of a gradient-based update
Batch / epoch	Examples for one update / one pass through the training set
Generalization	Useful performance on relevant new cases
Validation / test	Development comparison / assessment after choices are fixed
Underfitting / overfitting	Missing useful structure / fitting unhelpful training-specific patterns
Leakage	Information entering learning or selection that should be unavailable
Distribution shift	Data or relationships differ from the development setting
Regularization / early stopping	Constrain some forms of complexity / choose when to stop using validation

The data, calculations, diagrams, and exercises are original learning examples. Optional primary references:

Google: regression loss

Google: gradient descent

Google: overfitting

scikit-learn: preprocessing and leakage

Hugging Face: how transformers are trained

Next: Chapter 4 begins the programming foundations with Python for AI.